FaaSpot: Fault-Tolerant Serverless Workflow Execution on Spot Instances

Anonymous Author(s)
Submission Id: #81

Abstract

Serverless computing is increasingly used to run complex DAG workflows across diverse applications, and individual function execution times have grown to support them, pushing workflow makespans to hours. Open-source serverless platforms, deployed on a pool of VMs, typically use on-demand instances that grow costly over such long makespans. Spot instances are far cheaper, but a preemption is likely before such a workflow finishes. A preemption at a fan-out node or a task feeding a fan-in node stalls every parallel branch that shares it, a cascade that turns one failure into many. Prior systems address this challenge by migrating high-impact tasks to reliable, non-spot resources, sacrificing spot's cost advantage.

We present FaaSpot, a serverless workflow execution system that runs every workflow task on spot instances with no on-demand fallback. Our key insight is that the cascade-prone positions follow from the DAG's topology and can be identified before execution. Therefore, FaaSpot analyzes the workflow DAG once at submission time and classifies each task by the impact of its preemption. Selective checkpointing, survival-aware scaling, and lineage-based retry then act on this single classification, with no runtime coordination. We evaluate FaaSpot on a trace-driven spot emulator with eight DAG workflows, comparing its cost and performance against four state-of-the-art baselines. FaaSpot completes 89.03% of runs at $0.018 per successful run, 21pp above the average baseline success rate, while significantly reducing mean latency by 2.3×.

CCS Concepts

• **Computer systems organization** → **Cloud computing**; **Dependable and fault-tolerant systems and networks**.

Keywords

cloud computing, serverless computing, spot instances, DAG workflows, fault tolerance, checkpointing, adaptive scaling, workflow execution

ACM Reference Format:
Anonymous Author(s). 2026. FaaSpot: Fault-Tolerant Serverless Workflow Execution on Spot Instances. In *Proceedings of ACM Conference on XYZ (xyz'26)*. ACM, New York, NY, USA, 14 pages.

xyz'26, Woodstock, NY

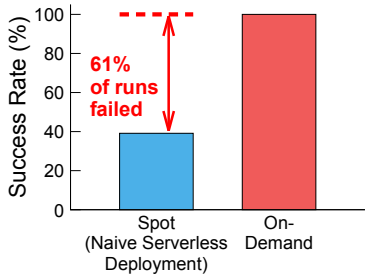


Figure 1: Average success rate of naive serverless deployment on spot vs. on-demand. *Results are averaged across our eight-workflows (500 runs each). Per-workflow results appear in Figure 2.*

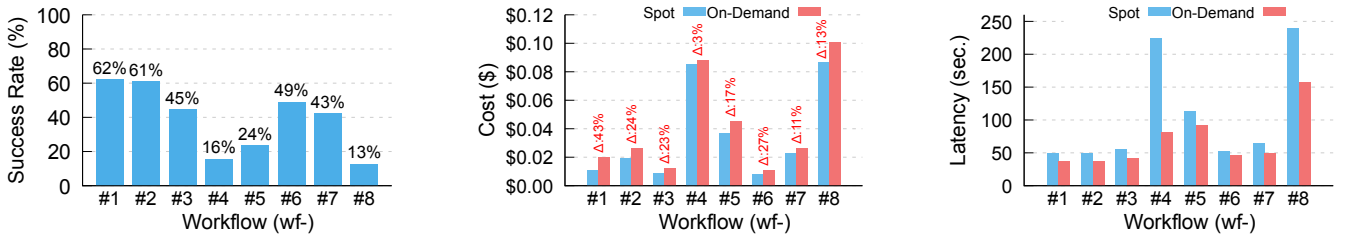
1 Introduction

Serverless computing is increasingly used to execute complex directed acyclic graph (DAG) workflows in data analytics [23, 31, 51], machine learning [1, 8, 24], and scientific computing [9, 34, 35]. These workflows are submitted, run to completion, and terminate, and they typically run repeatedly on a schedule or per experiment. To support such applications, major cloud providers now offer dedicated orchestration services [2, 14, 30], and individual function execution times have grown substantially. For example, Google Cloud now allows a single function to run for up to 60 minutes [13], while Azure has removed execution time limits for its current plans [29], compared to the few-minute limits of earlier generations. As a result, longer function chains and extended per-function runtimes push workflow makespans from tens of minutes to several hours.

Alongside these managed offerings, self-managed open-source serverless platforms, *e.g.*, Fission [12], OpenWhisk [3], are increasingly adopted, where operators run the serverless infrastructure on their own pool of VMs. These VMs are typically provisioned as on-demand instances, so infrastructure costs increase with workflow duration. Spot instances (2× – 10× cheaper [16, 42, 46]) become an attractive option for significantly reducing this cost. However, for workflows that run for tens of minutes to hours, spot instances are likely to be preempted before they finish [22, 46].

We measured the impact of spot preemptions on workflow completion. We deployed a set of serverless workflows naively on spot VMs, without any fault-tolerance mechanism. As shown in Figure 1, we observed that they completed only 39% of their runs on average, whereas on-demand deployments completed every run. The impact of a preemption is highly dependent on where it occurs in the workflow. For example, a preemption on a sequential *task*[1] (or *function*) affects only that task, whereas a preemption at a fan-out node or a task feeding a fan-in node can stall every parallel branch

[1]We use 'task' and 'function' interchangeably, since each task in a serverless workflow runs as a function.



(a) Success rate across workflows (b) Cost per successful run (c) Average end-to-end latency

Figure 2: Success rate, cost, and latency of naive serverless deployment on spot, compared to on-demand, across our eight-workflow suite. *500 runs per workflow on a trace-driven spot emulator (§5.2), using SkyPilot lifetime traces [46] for* `p3.2xlarge` *spot instances and SpotLake [22] pricing. (a) 'Success rate' falls significantly below 50% on the more complex workflows, dropping as low as ∼13%, while on-demand completes every run. (b) 'Cost-per-successful-run' stays below on-demand on every workflow, but the margin (Δ) is modest. Apart from the simplest workflow (43%), it stays below 30% and falls as low as 3%, far below spot's nominal price discount. (c) 'End-to-end latency on spot' exceeds on-demand by up to 2.7× because every failure forces a restart from the workflow's origin (the first function of the DAG).*

that shares it, a *cascade* that turns a single failure into many. As a result, a small number of structurally important positions are responsible for most of the workflow failures.

Fault tolerance on spot and transient resources has been studied mostly for non-serverless workloads [15, 37, 38, 41]. Most existing systems do not consider where preemptions occur in the workflow. They protect every task uniformly or react to preemptions at the VM level, without distinguishing a structurally critical task from an ordinary one (*e.g.*, a task on a sequential path). A few systems (*e.g.*, TR-Spark [47], Pado [49]) analyze the workload's DAG to protect the tasks whose preemption is most costly. But these systems all rely on reliable resources without preemption risk, migrating these high-impact tasks, those that can trigger a cascade, to stable storage or reserved nodes. Such use of non-spot resources (even if partial) reduces the spot's cost advantage. To fully leverage that advantage, as much of the workload as possible (ideally all of it) should run on spot. This direction leaves no reliable place to host or migrate these tasks. Therefore, the structural insight from above must be fully implemented for serverless workflows on spot, without the reliable resources that prior approaches assume.

To address this challenge, we present FaaSpot, a serverless workflow execution system that runs every workflow task on spot instances, with no on-demand fallback. FaaSpot's key insight is that positions (where a preemption would trigger a cascade) can be identified before execution begins, as they are determined by the DAG's topology rather than runtime state. FaaSpot analyzes the DAG once at submission time and classifies each task by the impact of its preemption. This single offline classification drives three online mechanisms: 1) selective checkpointing persists state only at high-impact positions; 2) survival-aware scaling provisions safer instances before a preemption is likely; and 3) lineage-based retry resumes a failed run from its nearest checkpoint rather than its origin. Because all three mechanisms are guided by the same classification, they consistently protect high-impact tasks, with no runtime coordination and no migration to non-spot resources. The result is spot-level cost with the reliability of on-demand execution.

We implement FaaSpot as a working prototype and develop a trace-driven spot emulator to evaluate it. Our workflow suite spans various application domains and a range of structures, from simple fork-join graphs to complex real-world pipelines [5, 10, 34, 36], including Epigenomics, LIGO, and Montage. Ensuring that FaaSpot and competing baseline systems receive identical spot preemption events is extremely challenging on live spot instances, so the emulator replays real spot lifetime and pricing traces at high fidelity. The emulator is also decoupled from FaaSpot, enabling any spot-aware system to be evaluated under the same preemption events. Across eight workflows and against four state-of-the-art spot fault-tolerance systems [18, 33, 43, 48], FaaSpot completes 89.03% of runs on average at $0.018 per successful run, 21pp above the average baseline success rate and at a lower cost per success than every baseline. Even against the strongest baseline (81.78% at $0.040), FaaSpot completes more runs at less than half the cost while reducing mean latency by 2.3×, outperforming all four on every evaluation metric.

This paper makes the following contributions toward fault-tolerant execution of serverless workflows on spot instances, *a largely underexplored area.*

- **FaaSpot**, a serverless workflow execution system that executes every serverless workflow task on spot instances without on-demand fallback (§3).
- **A unified failure-aware execution framework** that uses a single offline task classification to drive selective checkpointing, survival-aware scaling, and lineage-based retry, enabling consistent protection of high-impact tasks without runtime coordination (§4).
- **A trace-driven evaluation harness** that replays real spot lifetime and pricing traces, enabling fair and reproducible comparisons among spot-aware workflow execution systems under identical preemption events (§5.2, §6).

2 Motivation

2.1 Serverless Workflow Deployment on Spot

Spot instances are several times cheaper than on-demand [16, 42, 46], which makes them an attractive model for serverless DAG workflows. To quantify the actual cost benefits and reliability risk,

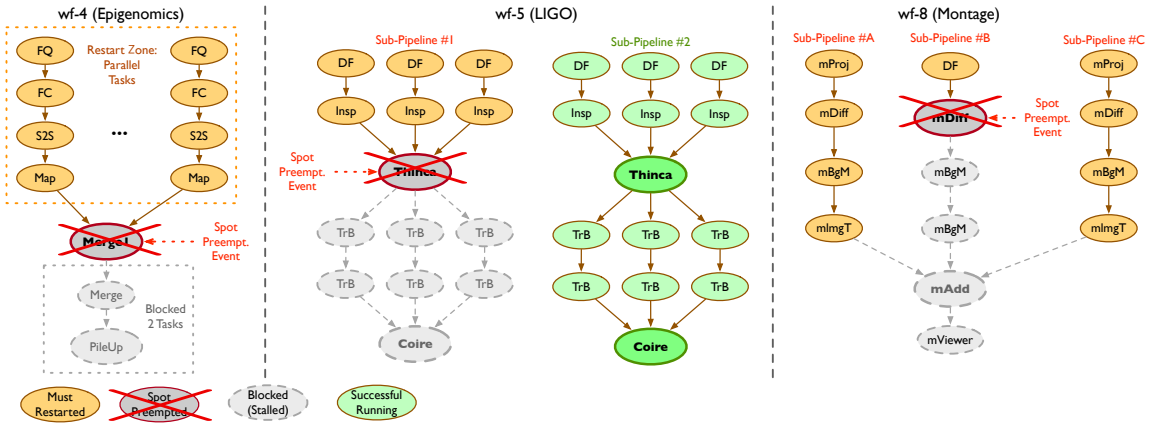


Figure 3: Three representative DAG workflows and the cascade failures at their synchronization barriers. *wf-4 (Epigenomics [5]), wf-5 (LIGO [5]), and wf-8 (Montage [5]) each contain fan-out nodes and synchronization barriers. Under naive deployment, a preemption of any parallel task feeding a barrier fails the entire run: the preempted task (red), every other branch that must restart with it (orange), and the downstream tasks left stalled at the barrier (gray). For wf-5, the two sub-pipelines contrast a preemption-hit branch with an unaffected one.*

we deployed a set of DAG workflows directly on spot instances using the serverless platform's default configuration, with no checkpointing, no retry coordination, and no migration to on-demand. We refer to this configuration as naive serverless deployment. A preemption anywhere in the workflow fails the entire run.

The measurements use a trace-driven spot emulator (described in §5.2). The emulator replays SkyPilot lifetime traces [46] for `p3.2xlarge` spot instances. Cost is computed using SpotLake [22] live spot pricing. Each workflow is run 500 times under matched trace conditions, allowing direct paired comparison against on-demand execution of the same workflow.

Figure 2 reports three measurements comparing naive spot deployment against on-demand across our eight-workflow suite[2]. Success rate of the naive deployment (Figure 2a) falls well below 50% on the more complex workflows, dropping as low as ~13%, while on-demand completes every run. Cost-per-successful-run (Figure 2b) stays below on-demand on every workflow (but only by a modest margin). End-to-end latency (Figure 2c) exceeds on-demand by up to 2.7× because every failure forces a restart from the workflow's origin (the first function).

The three measurements show spot VM's strengths and weaknesses. A cost-benefit remains, but it is modest. Spot is cheaper per successful run on every workflow, but except for the simplest one, the gap stays below 30% and falls as low as 3% (Figure 2b), far below the 2× to 10× discount reported in prior work [16, 42, 46]. Repeated restarts of failed workflows due to spot preemption diminish spot's price advantage. Furthermore, the increased latency and high failure rate are direct and substantial. This gap clearly indicates our opportunity. A fault-tolerant mechanism could recover spot's full price advantage while significantly reducing latency and failure rates to on-demand levels. The result would be a system that achieves spot-level cost with on-demand-level latency and reliability. The remainder of the paper develops such a mechanism.

[2]Three workflows are detailed in Figure 3, with the full suite described in §6.

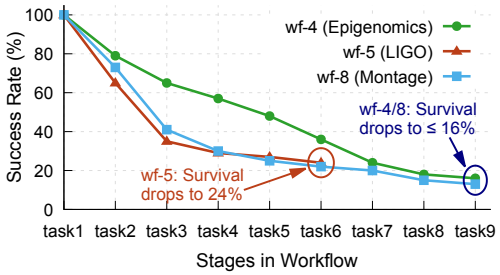


Figure 4: Stage-level success rate for three complex workflows under naive serverless deployment on spot. *All three workflows start near 100% at the first stage and decrease monotonically, with 13%–24% of runs surviving to the final stage.*

The next question is what makes complex workflows fail so much more often. §2.2 answers this question by analyzing structural patterns in the DAGs.

2.2 Why Complex Workflows Fail More Often?

The high failure rate on complex workflows (Figure 2a) cannot be explained by task count or execution time alone. The primary cause is dependency structure, specifically fan-out nodes and synchronization barriers. A preemption at a fan-out node blocks every parallel branch it spawns. A synchronization barrier is where all parallel branches must complete and merge at a fan-in node before downstream execution can continue, so a preemption of any task feeding the barrier stalls every other branch waiting at it. Three workflows exhibit both patterns, wf-4 (Epigenomics [5]), wf-5 (LIGO [5]), and wf-8 (Montage [5]), shown in Figure 3.

Figure 4 decomposes each workflow's success rate by stage, measured over 500 trials of naive deployment. All three workflows start near 100% at the first stage and decrease monotonically. However, only 13% to 24% of workflow runs survive at the final stage. The

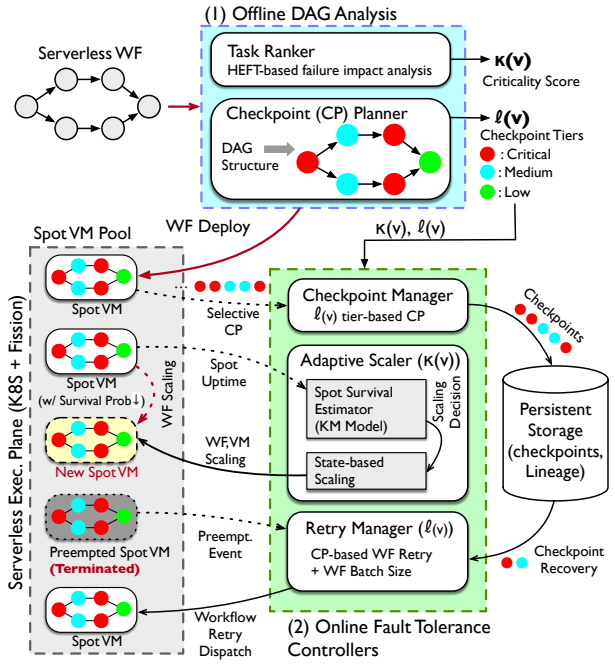


Figure 5: FaaSpot system architecture. *Offline phase (blue, (1)) derives a criticality score $\kappa(v)$ and checkpoint tier $\ell(v)$ from the workflow DAG. Three online fault-tolerance controllers (green, (2)) consume these signals at runtime (§4.2–§4.4). Execution plane (left) runs task pods on a shared spot pool via Fission on Kubernetes.*

steepest drops occur in the early stages, where the full population of runs is exposed to preemption across active parallel branches. wf-5 falls from 100% to 37% within three stages, and wf-8 falls to 42% over the same span. Each additional stage provides another opportunity for preemption to occur somewhere in the workflow, and any preemption can cause the entire run to fail. The cost is not uniform across DAG positions. A preemption on a sequential path forces only that task to recompute, but a preemption at a fan-out node or at a synchronization barrier propagates to every branch that shares it.

Complex workflows fail more often because they contain more high-impact positions. They have more fan-out nodes, wider parallel fan, and deeper synchronization barriers. Therefore, the same level of preemption activity produces a higher cumulative failure cost in complex workflows than in simple ones.

The pattern is not specific to any single application domain. wf-4 (Epigenomics), wf-5 (LIGO), and wf-8 (Montage) come from different application domains and have distinct DAG topologies. Yet all three suffer cascade failure rates that reflect similar structural patterns. The high failure rate is a property of DAG topology, not of any particular workload patterns. Because high-impact preemption positions are statically derivable from the DAG before execution begins, they can be selectively protected rather than uniformly.

3 FaaSpot System Overview

FaaSpot enables complex serverless workflows to run on spot instances by tolerating the cascade failures characterized in §2. FaaSpot achieves spot-level cost with on-demand-level reliability, without migrating any task off spot. This section overviews its architecture (§3.1) and the three design principles behind it (§3.2): *offline DAG analysis* to identify and classify high-impact preemption positions, *a criticality-annotated DAG* that encodes per-task structural risk and is shared across all fault tolerance decisions, and *tiered fault tolerance* that matches protection intensity to each task's cascade potential.

3.1 FaaSpot Architecture

Figure 5 shows the architecture of FaaSpot, which takes a serverless workflow DAG $G = (V, E)$ as input and executes it on spot instances through two (offline and online) coordinated phases.

The **offline phase** consists of two components. The *task ranker* (§4.1) computes a per-task criticality score $\kappa(v)$ that quantifies each task's downstream failure impact. The *checkpoint planner* (§4.1) classifies each task into three tiers $\ell(v)$ (`LOW`, `MEDIUM`, `CRITICAL` in Table 1) based on its structural position in the DAG. The criticality score $\kappa(v)$ and tier $\ell(v)$ are computed once before execution begins, then passed to all online controllers.

The **online phase** manages fault tolerance during execution through three controllers: the *checkpoint manager* (§4.2), the *adaptive scaler* (§4.3), and the *retry manager* (§4.4). All three make decisions based on the same $\kappa(v)$ and $\ell(v)$ from the offline phase.

FaaSpot runs on top of a serverless execution plane and persistent storage. The execution plane uses Fission [12], an open-source serverless platform, as the control plane and runs each task as a Kubernetes [44] pod on a shared spot VM pool (Figure 5, left). The persistent storage holds checkpoints and lineage records used by the online fault-tolerance controllers for recovery.

Scope and assumptions. FaaSpot runs every workflow task on spot with no on-demand fallback. However, the persistent storage and the Fission control plane are not on spot, and neither grows with the workflow. The design assumes that the DAG is fixed at submission, that a per-task execution time estimate is available.

FaaSpot execution flow. As the execution plane dispatches tasks, the checkpoint manager checks each task's tier $\ell(v)$. Checkpoints are written for tasks in higher tiers and skipped for those in the lowest tier to reduce overhead. Concurrently, the adaptive scaler monitors each spot instance's survival probability based on its uptime. When preemption risk increases, it scales out workflow tasks onto safer instances, provisioning new VMs if the safer-instance pool nears capacity. When a spot instance is preempted, the retry manager identifies the affected tasks and resumes each from its latest checkpoint rather than restarting from the workflow's origin. The adaptive scaler may concurrently provision additional resources to maintain forward progress. Execution continues until all workflow tasks complete.

3.2 Three Design Principles

We further elaborate on the three principles of FaaSpot, explaining why each design choice is made and how these choices reinforce one another.

Offline DAG analysis. Runtime identification of high-impact preemption positions often requires continuous analysis of the DAG as

Table 1: Tiered fault tolerance policy by task classification. *Each task's $\ell(v)$ (tier) determines whether its state is checkpointed and therefore whether a preempted run can resume from it.*

Tier	Structural Location	Ckpt.	Recovery Point
CRITICAL	Fan-out node, or task feeding a fan-in node	✓	Yes, preserves all branches past the barrier
MEDIUM	Parallel task that is neither a fan-out node nor feeds a fan-in node	✓	Yes, preserves its own branch
LOW	Sequential path, or fan-in node itself	✗	No, replayed during recovery

tasks execute, which can add latency to every scheduling decision. FaaSpot avoids this overhead by relying on a key property: fan-in and fan-out nodes are topological features that do not depend on runtime state. They can be identified once, before execution begins, and the result reused across all fault tolerance decisions.

Criticality-annotated DAG. The offline DAG analysis annotates each task with its criticality score $\kappa(v)$ and tier $\ell(v)$, producing a *criticality-annotated DAG*. This annotation is the only state the three online controllers share. Because every fault-tolerance decision is based on this same offline classification rather than on independently maintained runtime state, their decisions are consistent without any runtime coordination. §4 details how each controller consumes $\kappa(v)$ and $\ell(v)$.

Tiered fault tolerance. A uniform checkpoint policy treats all tasks equally, incurring I/O overhead on sequential tasks where re-execution cost is bounded by a single task. FaaSpot instead classifies tasks into three tiers based on the scope of cascade their preemption can trigger (Table 1). This classification allows the checkpoint manager to focus checkpoint I/O on high-impact locations and the retry manager to prioritize recovery for tasks with the highest downstream impact.

4 Failure-aware Serverless Workflow Execution

Building on the three design principles introduced in §3.2, we describe FaaSpot's fault-tolerant execution mechanisms for serverless workflows on spot instances. §4.1 describes the offline DAG analysis that produces the criticality score $\kappa(v)$ and checkpoint tier $\ell(v)$ before execution begins. The remaining three subsections describe the online fault-tolerance controllers: the checkpoint manager (§4.2), which writes checkpoints at tier-selected positions, the adaptive scaler (§4.3), which provisions safer VMs when survival probability drops, and the retry manager (§4.4), which resumes preempted workflows from their nearest persisted checkpoint. All components operate on the same criticality-annotated DAG produced by the offline analysis.

4.1 Offline DAG Analysis

Before execution begins, FaaSpot analyzes the workflow DAG using two components: the *task ranker* and the *checkpoint planner*. The task ranker computes a criticality score $\kappa(v)$, and checkpoint planner assigns a checkpoint tier $\ell(v) \in \{\texttt{CRITICAL}, \texttt{MEDIUM}, \texttt{LOW}\}$

Algorithm 1 Checkpoint Classification

Input: Tasks V, with succ(v) and minScale(v) for each $v \in V$
Output: Checkpoint levels $\ell(v) \in \{\texttt{CRITICAL}, \texttt{MEDIUM}, \texttt{LOW}\}$

```
for all v ∈ V do
    if ∃ s ∈ succ(v) : minScale(s) ≠ minScale(v) then
        ℓ(v) ← CRITICAL
    else if minScale(v) > 1 then
        ℓ(v) ← MEDIUM
    else
        ℓ(v) ← LOW
return ℓ
```

(Algo. 1). Both $\kappa(v)$ and $\ell(v)$ are computed once and passed to the online controllers (Figure 5).

Task ranker. The criticality score $\kappa(v)$ reflects both the longest remaining path from v and its structural position. A high $\kappa(v)$ means a preemption at v stalls many subsequent tasks, while a low $\kappa(v)$ means it affects only a few. FaaSpot builds on the Heterogeneous Earliest Finish Time (HEFT) algorithm, a standard priority metric in DAG scheduling that captures the longest remaining path from each task [45]. The base HEFT rank is formulated as:

$$\text{rank}(v) = w(v) + \max_{u \in \text{succ}(v)} \text{rank}(u), \tag{1}$$

where $w(v)$ is the execution time of v. The task ranker then adds two structural counts to *rank*(v), the number of branches v creates and the number stalling at the barrier v feeds, and min-max normalizes the sum across all tasks to obtain $\kappa(v) \in [0, 1]$. A fan-out node already carries a high *rank*(v) via the longest downstream chain, and the degree terms separate tasks whose remaining paths are comparable. The adaptive scaler (§4.3) uses only the resulting order to prioritize critical tasks during scale-out.

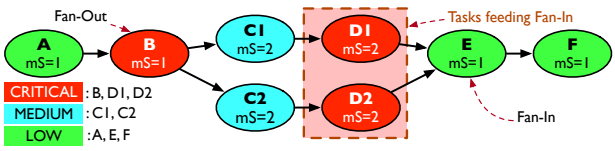


Figure 6: Three-tier $\ell(v)$ classification on an example DAG CRITICAL*: fan-out node (B) or tasks feeding a fan-in barrier (D1, D2, shaded).* MEDIUM*: parallel-branch interior (C1, C2).* LOW*: sequential or fan-in nodes (A, E, F).*

Checkpoint planner. This component assigns each task v a tier $\ell(v)$ (CRITICAL, MEDIUM, LOW) in the workflow DAG. Algo. 1 formalizes this classification using v's minScale. The minScale of v is the number of parallel tasks (functions) v requires. *i.e.*, 1 for a sequential task, or N for a task that runs as N parallel branches. CRITICAL tasks are fan-out nodes or parallel tasks feeding a fan-in node (v's successor has a different minScale). MEDIUM tasks have similar parallelism as their successors (minScale > 1, same as successors). LOW tasks lie on sequential paths or are fan-in nodes themselves (minScale = 1, same as successors). A change in minScale marks exactly the positions we identifed as cascade-prone (§2.2). Therefore, the tier signal captures where a preemption propagates across branches

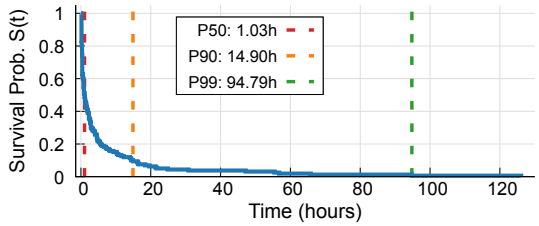


Figure 7: Conditional survival curve for p3.2xlarge spot instances in AWS region us-west-2a. *The survival probability $S(t)$ falls steeply within the first few hours, then tails off. FaaSpot's adaptive scaler maps each instance's $S(t)$ to a survival state (Table 2) and triggers scale-out before spot preemption.*

rather than where re-execution is merely expensive. Figure 6 illustrates the classification on an example DAG. The checkpoint planner computes $\ell(v)$ once offline, in parallel with the task ranker.

4.2 Selective Checkpointing

At runtime, when a task (a node/function in a single workflow) completes, the checkpoint manager checks its assigned tier $\ell(v)$ and decides whether to write a checkpoint to persistent storage. `CRITICAL` and `MEDIUM` tasks receive a checkpoint upon completion, while LOW tasks are skipped (Table 1). This selective checkpointing concentrates checkpoint I/O at cascade-prone positions in the DAG, where a persisted checkpoint recovers the most downstream work, and avoids the overhead of writing checkpoints at sequential tasks where re-execution cost is bounded by a single task.

4.3 Survival-aware Scaling

FaaSpot's survival-aware scaling is conducted by the adaptive scaler, which monitors the survival probability of spot instances and provisions additional spot VMs when the preemption risk increases. The adaptive scaler component combines three mechanisms: a Kaplan-Meier model estimates per-instance survival probability from observed uptime, a state-based policy determines the scaling factor, and HEFT ranks (computed offline by the task ranker, §4.1) determine the scale-out sequence, prioritizing critical tasks on at-risk spot instances.

KM (Kaplan-Meier) model. Figure 7 shows the conditional survival curve for `p3.2xlarge` spot instances in AWS region us-west-2a (from SkyPilot trace [46]). The probability falls steeply within the first few hours, then tails off. This time-dependent pattern motivates a survival-aware scaling policy. FaaSpot employs Kaplan–Meier (KM) estimator [19], a standard nonparametric method in survival analysis, to estimate per-instance survival probability from observed uptime. Spot preemption risk varies with uptime across instance types and regions, so no single distribution fits them all. The KM model instead builds the survival curve directly from each observed trace, without assuming any distribution.

For a spot instance with current uptime u, the KM model returns the probability that it survives an additional time window Δt:

$$S(\Delta t \mid u) = P(T > u + \Delta t \mid T > u), \tag{2}$$

Table 2: Survival states (0–3) and corresponding scaling factor ranges in the adaptive scaler. *Spot instances in the 'Safe (0) state' do not require scale-out (Algo. 2, lines 7–8); the other three states map $S(t)$ to a factor within the listed range.*

Survival State	Survival Prob.	Scaling Factor Range
0: Safe	$S(t) > 0.55$	1.0×
1: Proactive	$0.50 < S(t) \leq 0.55$	[1.1, 1.5]×
2: Warning	$0.40 < S(t) \leq 0.50$	[1.3, 1.8]×
3: Critical	$S(t) \leq 0.40$	[1.5, 2.0]×

where T is the instance's total lifetime. We write this conditional probability as $S(t)$ in the rest of the paper. $S(t)$ is looked up from the pre-computed curve in Figure 7. Spot instances are then classified into survival states (Table 2) based on $S(t)$. Spot instances in the safe state require no intervention, while instances in 'Proactive', 'Warning', or 'Critical' states are candidates for scale-out. Please note that the KM model is deferred until enough preemption events are observed to fit a stable curve.

State-based scaling. When a spot instance is classified into a non-safe survival state, the workflow running on the VM is at risk of preemption. FaaSpot mitigates this risk by scaling out the workflow onto safer instances. The remaining question is by *how much to scale out*. The amount of scale-out, the scaling factor, is determined by the survival state: a higher state (closer to preemption) maps to a larger scaling factor (Table 2). Each state defines a range, and within the range the factor scales based on $S(t)$ from the KM model. Because each additional replica incurs cost, this trade-off balances preemption risk against additional spending (due to scale-out operations). The factor f multiplies each currently running task's parallelism, and a task with minScale m is expanded to $\lceil f \cdot m \rceil$ replicas, distributed across safer VMs. For example, if a spot instance is in the '2: Warning' state ($f = 1.5$ within its [1.3, 1.8]× range), a task with $m = 4$ on the instance yields 6 replicas, which are deployed on other safer spot VMs. If the safer-VM pool approaches capacity, FaaSpot provisions additional VMs proactively, anticipating saturation from the current deployment plan. Algo. 2 details the scaling lookup and replica dispatch.

HEFT-ordered scale-out. Scaling resources are finite. To maximize workflow completion under spot preemption, FaaSpot creates a scale-out sequence for tasks on at-risk spot instances based on HEFT rank (§4.1). As a result, even if scale-out is interrupted by capacity saturation, `CRITICAL` tasks are more likely to have already received additional replicas. This HEFT-based ordering reduces the risk of cascading failures.

4.4 Lineage-based Retry

The retry manager recovers preempted workflow tasks by resuming each from its nearest upstream checkpoint rather than restarting from the workflow's origin. This manager replays only the sub-DAG downstream of that recovery point, then sizes the retry batch according to how far the current success rate falls below the target. Algo. 3 formalizes this decision loop.

Algorithm 2 Adaptive Scaler Decision Loop

Input: Spot instances I, uptime u_i for each $i \in I$

1: **procedure** AdaptiveScale(I)
2: **if** KM model not yet ready **then**
3: **return**
4: **for** each spot instance $i \in I$ **do**
5: $S \leftarrow$ KM.lookup(u_i) ▷ Survival Prob. from KM Model
6: $state \leftarrow$ classify(S)
7: **if** $state$ = Safe **then**
8: **continue**
9: $f \leftarrow$ lookup_factor($state, S$) ▷ in Table 2
10: $tasks \leftarrow$ sort_desc(tasks_on(i), key = HEFT rank)
11: **for** each task t in $tasks$ **do**
12: $m \leftarrow$ minScale(t)
13: $replicas \leftarrow \lceil f \cdot m \rceil$
14: deploy(t, $replicas$, target = safer instances)
15: **if** safer-instance pool is saturating **then**
16: provision_new_vms()

Table 3: Retry batch-sizing policy in the retry manager (Algo. 3). $\delta = \max(0,\ (\sigma_t - \sigma_c)/\sigma_t)$ *is the normalized gap between the current success rate* σ_c *and the operator-specified target* σ_t. B *is the number of candidates retried per cycle, and* α *is a multiplier applied when scale-out is inefficient. We empirically set* B_{high}, B_{mid}, B_{low}, *and* α *to 250, 200, 150, and 1.3.*

Condition	Batch Size	Effect
$\delta > \delta_{\text{high}}$	B_{high}	Aggressive recovery
$0 < \delta \leq \delta_{\text{high}}$	B_{mid}	Moderate recovery
$\delta = 0$	B_{low}	Maintenance only
Scale-out ineffective for ≥ 2 cycles	$B \times \alpha$	Shift effort from scaling to retrying

Checkpoint-based recovery. When a task is preempted, the retry manager identifies the recovery point by traversing the DAG backward from the failed task. Because checkpoints are written upon task completion (in §4.2), the recovery point is the nearest completed predecessor whose tier guarantees that a checkpoint has been persisted. Execution resumes from that point, replaying only the sub-DAG downstream of the recovery point rather than the full workflow. Each workflow run is subject to a maximum retry limit $L_{\max}$ (Algo. 3, line 2), set between 12 and 35 by workflow size. A run that uses all its retries without completing is counted as a failure, which is why success rates stay below 100%.

Retry batch sizing. The checkpoint-based recovery (described above) determines how to resume a single failed run. In practice, preemptions in a shared spot pool can leave multiple workflow runs incomplete simultaneously. However, immediately retrying all incomplete runs is undesirable, as they will compete for the same limited spot capacity. Therefore, the retry manager selects a subset of candidates to retry each cycle. From all incomplete runs, only those below their maximum retry limit are eligible. Each eligible

Algorithm 3 Lineage-based retry

Input: Incomplete workflow runs U, retry history, workflow success rates (σ_t, σ_c)
Output: Batch of runs retried from latest checkpoints

// Compute success gap
1: $\delta \leftarrow \max(0,\ (\sigma_t - \sigma_c)\ /\ \sigma_t)$
// Filter and rank candidates
2: $C \leftarrow \{u \in U : \text{incomplete}, R[u] < L_{\max}[u]\}$
3: **for** $u \in C$ **do**
4: $p(u) \leftarrow \pi_w + \lfloor \delta \times \lambda_d \rfloor + (n_c \times w_c) - (r \times w_r)$
5: Sort C descending by $p(u)$
// Set batch size (Table 3)
6: **if** $\delta > \delta_{\text{high}}$ **then** $B \leftarrow B_{\text{high}}$
7: **else if** $\delta > 0$ **then** $B \leftarrow B_{\text{mid}}$
8: **else** $B \leftarrow B_{\text{low}}$
// Enlarge if scale-out is ineffective
9: **if** scale-out has not improved σ_c for ≥ 2 cycles **then**
10: $B \leftarrow \lfloor B \times \alpha \rfloor$
// Retry from latest checkpoint
11: **for** top B runs from sorted C **do**
12: Resume from latest checkpoint
13: Increment retry count

workflow run u receives a priority score:

$$p(u) = \pi_w + \lfloor \delta \times \lambda_d \rfloor + (n_c \times w_c) - (r \times w_r), \tag{3}$$

where π_w is a per-workflow base priority, λ_d is an urgency scaling constant, n_c is the number of tasks already completed in u, w_c is the per-task completion weight, r is the number of prior retries for u, w_r is the per-retry penalty weight, and δ is the normalized success-rate gap defined in Table 3.

The score increases with the number of completed tasks and decreases with the number of prior retries, so that near-complete runs are retried first. Tier enters this ordering only indirectly, since resuming from a higher-tier checkpoint preserves more completed tasks. Because each retry queue holds runs of one workflow, normalizing n_c by workflow size would not change the retry order. The operator specifies a target success rate σ_t for the workflow, and the retry manager uses the gap δ between σ_t and the current rate σ_c to control the batch size (Table 3). Therefore, a large gap triggers aggressive recovery, while a small or zero gap reduces the batch to conserve spot capacity (Algo. 3, lines 6–8). When the recent scale-out operations of the adaptive scaler (§4.3) fail to improve the success rate for two or more consecutive cycles, the retry manager enlarges the batch further, shifting recovery effort from scaling to retrying (Algo. 3, lines 9–10).

5 Implementation

5.1 FaaSpot Implementation

FaaSpot is implemented in approximately 17.5K lines of Python on a Kubernetes cluster using Fission as the serverless execution platform. Fission was selected over other open-source FaaS frameworks for its native workflow support and ease of deployment. All runtime state (*e.g.*, checkpoint records, retry history, and scaling decisions) is persisted in MySQL. Checkpoint state is written to the same store,

which runs on a non-spot VM so that a preemption cannot erase it. Writes are transactional, so a failure during a checkpoint write leaves the previous checkpoint intact and recovery falls back to it. The offline analysis outputs, $\kappa(v)$ and $\ell(v)$, are distributed to online controllers via per-task Kubernetes `ConfigMaps` populated before workflow deployment.

Task ranker runs as a standalone process at deployment time. This component parses the workflow DAG definition (`tasks.json`), computes $\kappa(v)$ for each task, and writes the result to a local file used by the checkpoint planner.

Checkpoint planner runs after the task ranker completes. This planner reads $\kappa(v)$ and the DAG definition, assigns $\ell(v)$ for each task, and populates per-task Kubernetes `ConfigMaps` via the cluster API. At runtime, the checkpointing mechanism embedded in each task reads $\ell(v)$ from its attached `ConfigMap` and writes task output state to MySQL at the configured interval.

Adaptive scaler runs as an independent subprocess on the cluster control plane. At startup, this component generates a KM survival curve from spot preemption traces and loads it into memory. At each polling interval, it retrieves active pod uptimes, evaluates survival probability against the curve, and issues scale-out operations prioritized by $\kappa(v)$. Scaling decisions are recorded in MySQL for coordination with the retry manager.

Retry manager runs as an independent subprocess alongside the scaler. This manager polls MySQL at regular intervals to detect failed tasks and dispatches recovery requests to the Fission gateway. Prior retry history is read at startup to calibrate batch sizing, and retry decisions are persisted to the same store.

5.2 Trace-driven Spot Emulator

Evaluating FaaSpot on live cloud spot instances is impractical for two reasons. Spot market conditions shift continuously, so back-to-back runs of different systems face different interruption sequences, making a fair paired comparison impossible. Sustained evaluation across multiple workflows and methods at scale is financially expensive and time-consuming.

Therefore, we developed a trace-driven spot emulator to fairly evaluate the performance of FaaSpot and competing approaches. The emulator is designed around three goals. *1) Trace fidelity*: we used spot preemption events that are derived from real production traces. The emulator replays empirical lifetime distributions collected from spot markets [22, 46]. *2) Fair evaluation*: all methods under comparison receive identical spot preemption events within each trial, so reported differences reflect controller quality rather than differences in the underlying spot conditions. *3) Controller-agnostic design*: we developed the emulator to be decoupled from FaaSpot and expose an evaluation API, which allows any spot-aware system to plug in as an evaluation target.

Spot traces. The emulator uses two public trace sources. Spot instance lifetimes are derived from the SkyPilot availability traces [46], which record uptime across multiple spot VMs over a two-week period with different regions, operating systems, and instance types. The emulator fits an empirical lifetime distribution from these traces. Figure 7 (in §4) shows the resulting survival curve for `p3.2xlarge` VMs in AWS us-west-2a region. Spot pricing is obtained from SpotLake [22], which archives historical spot prices across regions and instance types. The emulator applies these prices to compute per-VM execution cost over each VM's observed lifetime.

Spot emulator architecture. The emulator is composed of four components. The *distribution extractor* derives an empirical lifetime distribution from the raw traces, filtered by region, operating system, and instance type, as lifetime characteristics vary significantly across these dimensions [39, 40, 46]. The *lifecycle manager* emulates the lifecycle of each spot VM. When a new VM is provisioned, it samples a lifetime between p_{10} and p_{90} of the derived distribution, excluding extreme outliers at both tails. The manager runs at 60× accelerated time, enabling large-scale workflow evaluation without the time and cost overhead of live spot experiments. It tracks each VM's uptime on this accelerated clock and triggers a preemption event when the uptime reaches the assigned lifetime. The *cost calculator* computes the execution cost for each VM using the historical spot prices obtained from SpotLake. When a VM is preempted or completes its workload, the calculator records the accrued cost over the VM's observed lifetime. The *preemption dispatcher* provides the interface between the emulator and the execution plane. When a VM's uptime reaches its assigned lifetime, the dispatcher deletes the affected instance, terminates all co-located tasks, and notifies the cost calculator to record the accrued cost.

Execution model. The emulator runs on a physical server cluster that serves as an emulated data center. Each physical machine hosts a VM allocated with dedicated cores and memory, representing a single physical server in the emulated environment. Kubernetes runs across these VMs, and each Kubernetes pod represents a spot VM in the emulation. The lifecycle manager assigns each pod a lifetime sampled from the derived distribution and preempts the pod when its uptime reaches the assigned lifetime. Serverless workflow tasks are deployed as containers inside these pods via Fission, so workflows execute on real infrastructure under emulated spot preemption conditions. A preemption event terminates all containers in the affected pod, matching the behavior of real cloud providers where instance-level preemption affects all co-located tasks and workflows. Each trial assigns pod lifetimes from a fixed seed, so all methods under comparison receive identical interruption sequences within each paired trial.

6 Evaluation

We evaluate FaaSpot on the trace-driven spot emulator (§5.2) with eight structurally diverse DAG workflows and four baselines. The key results are:

- FaaSpot achieves 89.03% average success rate at $0.018 cost per success, outperforming all four baselines on both metrics simultaneously (§6.2).
- FaaSpot's mean latency of 126s is 2.3× lower than the nearest competitors, with the smallest cross-workflow variance at every percentile (§6.3).
- FaaSpot writes 10.6 checkpoints per successful run, while baselines write 1.17× to 3.8× more, because it places checkpoints selectively at cascade-prone positions (§6.4).
- Ablation confirms that all three fault-tolerance mechanisms are essential, as removing any one increases cost per success by 1.8× to 4.6×, and removing lineage-based retry reduces the success rate to 50% (§6.5).

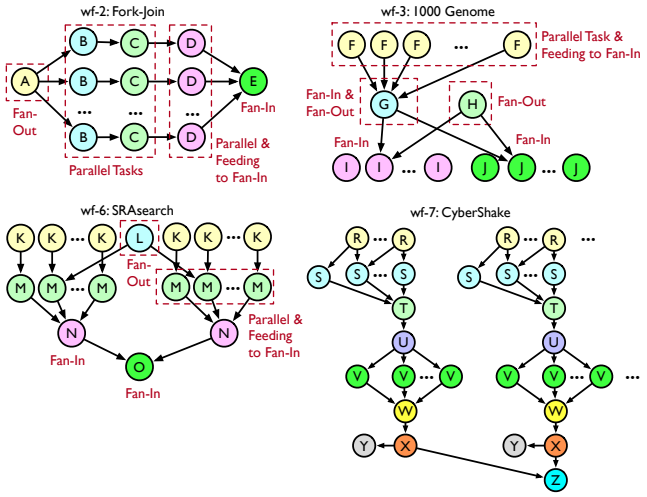


Figure 8: DAG structures of four additional workflows used in evaluation. *'wf-2': a fork-join DAG, 'wf-3': 1000Genome [10, 34], 'wf-6': SRAsearch [34, 36], and 'wf-7': CyberShake workflow [5].*

6.1 Evaluation Setup

Serverless workflow suite. We use eight DAG workflows drawn from four application domains. Three workflows with high cascade potential (wf-4, wf-5, wf-8) were introduced in §2 (Figure 3). Figure 8 shows four additional workflows: wf-2, a fork-join DAG with a single fan-out/fan-in pair; wf-3 (1000Genome [10, 34]), a pipeline with large fan-out/fan-in stages; wf-6 (SRAsearch [34, 36]), a two-branch DAG with parallel tasks per branch; and wf-7 (CyberShake [5]), a wide DAG with repeated fan-out/fan-in stages. The remaining workflow, wf-1, is a compact DAG with a single fan-out and no synchronization barrier.

Baselines. FaaSpot is evaluated against four baselines. Snape [48] dynamically adjusts the ratio of spot to on-demand instances using eviction prediction and constrained reinforcement learning. Hourglass [18] manages temporal slack as a risk budget for time-constrained graph-processing jobs, falling back to on-demand instances when slack is exhausted. MScheduler [33] recovers spot-preempted HPC jobs through application-native restart mechanisms and cost-based instance selection across regions. Burst-HADS [43] schedules independent bag-of-tasks workloads across spot and burstable instances with deadline constraints. None of the four baselines incorporates DAG structure into checkpoint placement or retry ordering.

Experiment configuration and metrics. Each workflow is executed 500 times per method per trial, and each experiment is repeated over 5 trials. FaaSpot and four baselines receive identical spot preemption sequences within each trial to ensure that performance differences reflect controller behavior rather than differences in interruption conditions. All methods share the same DAG definitions, resource pool, and spot prices. The experiments use three spot instance types (`m3.2xlarge`, `p3.2xlarge`, `t3.2xlarge`) across two AWS regions (us-west-2a, us-west-2b).

We report three metrics. Success rate is the fraction of submitted runs that complete. Cost per success is the total cost divided by the number of completed runs. Note that the reported cost covers compute only. The shared store runs as a single VM serving every method, so its cost does not scale with checkpoint frequency and will not change the relative comparison. Average latency is the mean end-to-end wall-clock time over completed runs, including queueing, retries, and checkpoint replay. Serverless function startup overhead is also included in this latency metric, as each task runs as a real pod on Kubernetes. Latency values are used for relative comparison only, as absolute values reflect the emulated timeline.

Table 4: Average success rate and cost per success across the eight test workflows. *Best result per column in bold.*

Method	Success Rate (%)	Cost/Succ. Run ($)
FaaSpot	**89.03 ± 5.87**	**0.018 ± 0.007**
Snape	58.56 ± 6.17	0.050 ± 0.011
Hourglass	76.22 ± 8.39	0.047 ± 0.011
MScheduler	81.78 ± 7.95	0.040 ± 0.007
Burst-HADS	53.43 ± 7.74	0.259 ± 0.071

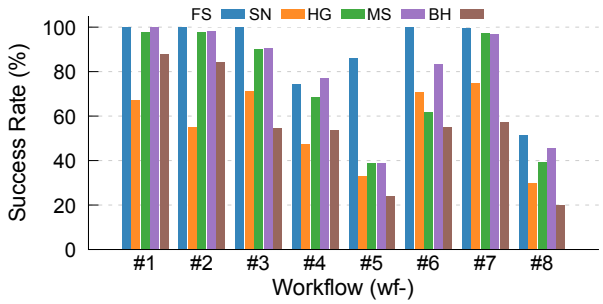
6.2 Success Rate and Cost Efficiency

We report the workflow success rate and cost per successful run together, since high completion is meaningful only if it does not require excessive VM execution cost. Figure 9 shows both metrics for FaaSpot and all four baselines across the eight workflows: Figure 9a for success rate and 9b for cost per successful run.

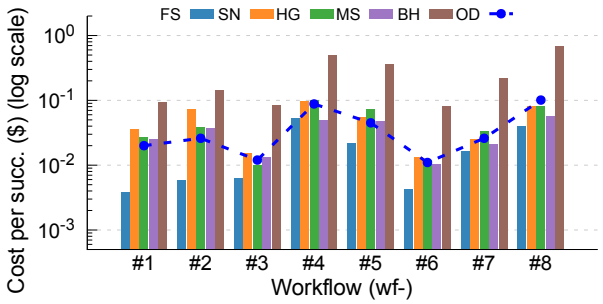
FaaSpot outperforms all four baselines on both average success rate and cost per success. It achieves 89.03% average success rate at only $0.018 cost per success. The closest competitor, MScheduler, reaches 81.78% at $0.04, which is 7.25 pp lower in success rate and more than 2× higher in cost per completed run. Hourglass reaches 76.22% at $0.047, Snape 58.56% at $0.05, and Burst-HADS 53.43% at $0.259 (Table 4).

The differences in success rate and cost vary with workflow DAG structure. We observe that, on wf-4, MScheduler surpasses FaaSpot by 2.5 pp on success (77.1% vs. 74.6%), but at $0.067 per success vs. FaaSpot's $0.055 (22%↓). On wf-5, FaaSpot reaches 86.2% while all four baselines fall below 41%. Figure 10 plots total cost against success rate for these two high-cascade workflows. We omit the remaining workflows for brevity, but they show a similar pattern.

The performance gap is primarily due to the four baselines not considering the DAG structure when making fault-tolerance decisions. They treat a preemption on a sequential path and one at a cascade-prone position identically. In contrast, FaaSpot's offline tier assignment (§4.1) distinguishes these cases before the first task executes, placing checkpoints only at cascade-prone positions. When a preemption occurs at a synchronization barrier, recovery focuses on the sub-DAG downstream of the nearest checkpoint rather than requiring a full-workflow restart. Therefore, recovery cost is concentrated on tasks classified as `CRITICAL` or `MEDIUM` by $\ell(v)$ rather than being spread uniformly, which is why FaaSpot achieves higher success at lower cost than methods that treat all tasks equally.



(a) Workflow success rate



(b) Cost per successful run

Figure 9: Success rate and cost per success for each workflow and baseline. *(a) Success rate across eight workflows: FaaSpot achieves the highest success rate on seven of eight workflows, with the largest margin on wf-5 (+43 pp over the nearest baseline). On-demand completes every run. (b) Cost per successful run (log scale, lower is better): FaaSpot achieves the lowest cost per success on seven of eight workflows. The dashed line marks on-demand cost per successful run (Figure 2b) 'FS': FaaSpot, 'SN': Snape, 'HG': Hourglass, 'MS': MScheduler, and 'BH': Burst-HADS.*

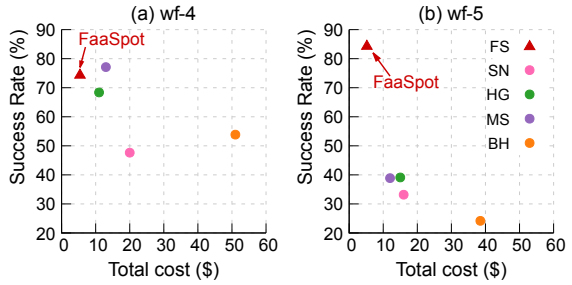


Figure 10: Total cost vs. success rate for wf-4 and wf-5. *The top-left region indicates higher success at lower cost. On wf-4, FaaSpot (red triangle) has the lowest cost while MScheduler reaches slightly higher success at over twice the cost. On wf-5, FaaSpot leads on both axes with all four baselines clustering at higher cost and lower success.*

6.3 End-to-end Latency

We next report end-to-end latency, and Table 5 shows the mean, p50, p95, and p99 latency for each method. FaaSpot achieves the lowest latency across all four percentiles. Its mean of 126s is 2.3× lower than the nearest competitors, which are Snape (297s) and Burst-HADS (290s). The latency difference is especially large at the tail. At p95, FaaSpot achieves 245s, compared to 704s for Snape, 635s for MScheduler, and 1310s for Burst-HADS. At p99, the gap increases further, with Hourglass reaching 2199s and Burst-HADS 1897s against FaaSpot's 346s.

Beyond absolute values, FaaSpot also shows the smallest variance across workflows. Its ± values in Table 5 are consistently the lowest (*e.g.*, ±42 at p50, compared to ±114 for Snape and ±143 for Hourglass), indicating that FaaSpot delivers predictable latency. The baselines show high variance because their recovery cost depends on where in the DAG the preemption happens to land. In contrast, FaaSpot's lineage-based retry (§4.4) reduces this variance by always resuming from the nearest upstream checkpoint rather than restarting from the workflow origin.

Table 5: End-to-end latency across the eight test workflows (sec). *Each column reports the workflow-averaged percentile over completed runs, with ± indicating cross-workflow variation. Best value per column in bold.*

Method	Mean	p50	p95	p99
FaaSpot	**126 ± 47**	**110 ± 42**	**245 ± 85**	**346 ± 108**
Snape	297 ± 108	264 ± 114	704 ± 223	799 ± 237
Hourglass	506 ± 139	443 ± 143	1138 ± 339	2199 ± 219
MScheduler	334 ± 128	326 ± 130	635 ± 237	680 ± 250
Burst-HADS	290 ± 66	115 ± 64	1310 ± 158	1897 ± 209

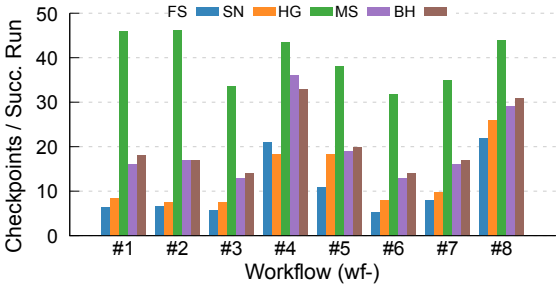


Figure 11: Average number of checkpoints per successful run for each workflow. *Lower values indicate more selective checkpoint placement. FaaSpot achieves the fewest checkpoints per success on every workflow.*

6.4 Checkpoint Efficiency

We next report checkpoint efficiency by measuring the number of checkpoints each method requires for successful completion. A lower count indicates that the method places checkpoints more selectively, reducing I/O overhead without sacrificing reliability. Figure 11 reports the average number of checkpoints per successful run for each method across the eight workflows.

FaaSpot completes workflow executions with an average of 10.6 checkpoints per successful execution, which is the fewest among all

Table 6: Contribution of each mechanism. *Average success rate and cost per successful run for FaaSpot and three ablated variants, each removing one mechanism. Best value per column in bold.*

Variant	Avg. Succ. (%)	Avg. Cost/Succ. ($)
FaaSpot	**89.03**	**0.018**
A (w/o Select. Checkpoint)	86.44	0.033
B (w/o Adapt. Scaling)	86.74	0.041
C (w/o Lineage Retry)	50.05	0.082

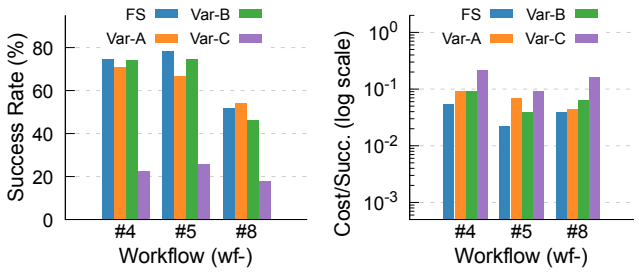
evaluated methods. Snape requires an average of 12.4 checkpoints per successful workflow execution (1.17× more), MScheduler requires 19.6 (1.8× more), and Hourglass incurs 39.9 checkpoints (3.8× more). Among the baselines, Snape requires the fewest checkpoints per success (12.4 vs. FaaSpot's 10.6), but this is because Snape partially uses on-demand instances, so fewer tasks are exposed to preemption in the first place. None of the baselines selects checkpoint positions based on the DAG structure. Hourglass checkpoints every task uniformly, which is the main reason for its highest checkpoint count (39.9) among all methods. FaaSpot's tier assignment $\ell(v)$ concentrates checkpoints at fan-out nodes and tasks feeding fan-in nodes, the positions where a missed checkpoint forces the most downstream re-execution. This selective checkpointing explains why FaaSpot writes the fewest checkpoints while still maintaining the highest success rate among all methods.

6.5 Ablation Study

To isolate the contribution of each mechanism, we evaluate three ablated variants of FaaSpot. Each variant removes one mechanism while keeping the others. 'Variant-A' removes selective checkpointing and instead checkpoints every task uniformly, as in systems that do not consider DAG structure. 'Variant-B' removes survival-aware scaling, so tasks are recovered through checkpointing and retry alone, without proactive scale-out onto safer instances. 'Variant-C' removes lineage-based retry. As checkpoints are used exclusively for lineage-based recovery, checkpointing is also disabled. Consequently, a preempted workflow is restarted from its origin. We compare all three against full FaaSpot on the eight-workflow suite, reporting success rate and cost per successful run.

Table 6 reports the average success rate and cost per successful run for each variant across the eight workflows. The three variants reveal a consistent pattern. Removing selective checkpointing (Variant-A) or survival-aware scaling (Variant-B) causes only a modest reduction in success rate (86.44% and 86.74%, respectively, vs. 89.03% for FaaSpot), but substantially increases the cost of achieving a successful run. In contrast, removing lineage-based retry (Variant-C) reduces the success rate to 50.05% and increases the cost per successful run to $0.082.

The three variants isolate where each mechanism's benefit comes from. Variant-A checkpoints every task and achieves a success rate similar to that of FaaSpot. However, many of these checkpoints occur at sequential positions where re-execution is inexpensive, increasing the cost to $0.033. This result shows that selective checkpointing improves efficiency not by increasing the success rate, but by avoiding checkpoints where the benefits of recovery are small. Variant-B recovers without survival-aware scaling, so recovery relies entirely on checkpointing and retry. As a result, more tasks must be re-executed after spot preemptions, which naturally increases the cost to $0.041, despite a success rate close to FaaSpot's. Variant-C shows the largest degradation. Without checkpoint-based recovery, every preemption forces the workflow to restart from its origin, reducing the success rate to 50.05% and increasing the cost to $0.082. These results show that lineage-based retry is the primary contributor to recovery effectiveness, and both selective checkpointing and survival-aware scaling improve the efficiency of that recovery. Together, the three mechanisms enable FaaSpot to achieve both high completion and low cost on spot instances.



(a) Workflow success rate **(b) Cost per successful run**

Figure 12: Ablation results on the three high-cascade workflows (wf-4, wf-5, wf-8). *(a) Success rate and (b) cost per successful run (log scale) for FaaSpot (FS) and three variants: Var-A (w/o selective checkpointing), Var-B (w/o survival-aware scaling), and Var-C (w/o lineage-based retry).*

We now take a closer look at the three high-cascade workflows: wf-4, wf-5, and wf-8. The results are shown in Figure 12. Removing checkpoint-based recovery (Variant-C) shows the largest degradation here, reducing success rates to 22.7%, 25.9%, and 17.9%, respectively. This degraded performance is mainly due to each preemption discarding all prior progress, forcing these long, highly parallel workflows to restart from their origins. On wf-8, where every variant performs poorly, Variant-A achieves a slightly higher success rate than FaaSpot (54.2% vs. 51.7%) because checkpointing every task recovers a few additional runs. However, this improvement comes at a higher cost per successful run ($0.044 vs. $0.040). This is the trade-off selective checkpointing is designed to make. FaaSpot skips checkpoints where the benefit is small, accepting a slightly lower completion rate in return.

6.6 Parameter Sensitivity

In our evaluation, we used the 'safe boundary' and 'batch sizes' specified in Tables 2 and 3. We test how sensitive the results are to both parameters, using wf-4, 5, and 8. Table 7 reports the results.

The results confirm that the survival-state boundaries significantly affect both metrics. We shifted all three boundaries down together so that the scaler intervenes at a lower survival probability. Moving this safe boundary from 0.55 to 0.45 degrades the success rate on wf-8 from 51.7% to 27.0% and doubles the execution cost.

A lower boundary means the scaler acts later, so a preemption is more likely to have already occurred. Increasing the survival

Table 7: Sensitivity to the survival state boundaries (Table 2) and the retry batch size (Table 3). *For the boundaries, all three shift together and the Safe boundary is shown. Each cell reports the success rate (%) and cost per successful run ($) for wf-4/5/8 in Figure 3 (in §2). We used the default settings (in bold) in our evaluation.*

Configuration	wf-4	wf-5	wf-8
Survival state boundaries			
Safe at $S(t)$ = 0.45	50.2% / $0.12	56.7% / $0.05	27.0% / $0.14
Safe at $S(t)$ = 0.50	62.8% / $0.08	69.2% / $0.05	36.0% / $0.09
Safe at $S(t)$ = 0.55	**74.6% / $0.06**	**86.2% / $0.04**	**51.7% / $0.07**
Retry batch size: (B_{high}, B_{mid}, B_{low})			
(150, 100, 50)	68.6% / $0.08	77.5% / $0.06	57.0% / $0.10
(250, 200, 150)	**74.6% / $0.06**	**86.2% / $0.04**	**51.7% / $0.07**
(350, 300, 250)	72.2% / $0.07	91.2% / $0.05	49.6% / $0.11

boundaries raises the success rate. However, this change naturally increases cost per successful run, since replicas are added to instances that were less likely to be preempted. The default setting therefore balances these two cases, aiming for the scaler to act early enough to prevent cascades without wasting capacity.

We also observe that the retry batch size has a smaller effect. The default setting offers the lowest cost per successful run on all three workflows. Its effect on success rate depends on the workflow. For example, a larger batch helps wf-5 but hurts wf-8, and a smaller batch does the opposite. We therefore selected the values (B_{max}=250, B_{mid}=200, B_{low}=150) that give the lowest cost per successful run.

7 Limitations

We identify five limitations of the current evaluation and our system design for FaaSpot. First, all experiments in this paper are conducted on a trace-driven spot emulator, not on live cloud infrastructure. The emulator ensures fair comparison by exposing all methods to identical preemption events, but the results reflect relative performance differences between methods rather than absolute production-level measurements.

Second, checkpoint I/O latency is not modeled in the emulator. Since FaaSpot writes fewer checkpoints than all evaluated baselines (§6.4), incorporating this overhead would likely negatively affect methods with higher checkpoint frequency, further strengthening FaaSpot's advantage.

Third, the offline tier assignment assumes that the workflow DAG structure is fully known at submission time. Some scientific workflows have data-dependent branching where execution paths vary across runs (*e.g.*, DayDream [35]). Such workflows fall outside our current scope. Extending the offline analysis with online DAG refinement is a natural direction for future work.

Fourth, the task ranker requires a per-task execution time estimate. Inaccurate estimates can shift the criticality score, though the scaler uses only the resulting ranking. Using this same estimate as an additional tier signal, so that a long-running task on a sequential path can be promoted, is our future work.

Finally, the evaluation uses three instance types (`m3.2xlarge`, `p3.2xlarge`, `t3.2xlarge`) across two AWS regions (us-west-2a/2b). Spot preemption-rates differ across regions and instance families. Their distribution shape also depends on the underlying availability mechanism. Therefore, the KM survival estimation model and scaling thresholds (Table 2, §4.3) may require recalibration for deployment targets. This recalibration affects only the adaptive scaler, as checkpointing and retry depend on DAG structure.

8 Related Work

FaaSpot intersects two areas of prior work. The first is *fault-tolerant execution on spot instances*, where checkpoint placement, resource provisioning, and scheduling have been studied extensively for batch and workflow jobs running on preemptible instances. The second is *serverless workflow execution*, where DAG dispatch frameworks and platform-level optimizations have matured but have not been combined with spot preemption tolerance. FaaSpot operates at the intersection of these two areas, applying DAG-structure-aware fault tolerance to serverless workflows running on spot instances without an on-demand fallback.

Fault-tolerant execution on spot instances. Checkpoint-based fault tolerance has a long history, with foundational work establishing the optimal checkpoint interval as a function of overhead and expected failure cost [50]. The problem has been studied extensively in the context of spot and transient cloud resources, where instances can be revoked at any time with short notice. Prior work spans task replication strategies [32], spot market characterization [4, 39, 46], and trace-driven availability modeling [20, 22, 40], and subsequent systems build on these efforts.

Building on this foundation, several systems address spot preemption at the resource management level. Hourglass [18] manages temporal slack as a risk budget for time-constrained graph processing jobs, falling back to on-demand instances when slack is exhausted. Snape [48] dynamically adjusts the ratio of spot to on-demand instances using eviction prediction and constrained reinforcement learning. Burst-HADS [43] schedules independent bag-of-tasks workloads across spot and burstable instances with deadline constraints. MScheduler [33] leverages application-native restart mechanisms to recover single HPC jobs from spot revocation with cost-based instance selection across regions. SpotVerse [41] reduces cost by selecting cloud regions based on interruption frequency and placement score, but operates at the granularity of whole workflows and cloud regions rather than the internal DAG structure. TR-Spark [47] addresses cascading re-computation in Spark's stage-based DAG model through lineage-aware checkpointing and resource-stability-aware scheduling, demonstrating that structure-aware checkpoint placement substantially outperforms uniform policies on transient resources. Pado [49] likewise analyzes the job DAG, identifying operators whose eviction would trigger the most recomputation and placing them on eviction-free reserved nodes instead of checkpointing them. More broadly, Flint [37] and portfolio-driven management [38] adapt batch analytics to transient server pools, and Tributary [15] provides SLO-constrained elastic control under spot preemption.

Most prior systems optimize checkpoint placement per task without reference to downstream dependencies in the workflow DAG. Their provisioning decisions do not distinguish between a preemption on a sequential path and one at a fan-out node feeding a synchronization barrier. TR-Spark and Pado are the closest to

FaaSpot in exploiting DAG structure to protect the positions whose loss is most costly. However, TR-Spark operates within Spark's synchronous stage model, and Pado depends on reserved nodes that are always available. Neither targets serverless workflows running on spot instances without an on-demand fallback, where tasks must be protected through checkpointing, scaling, and retry rather than placement on safe nodes.

Serverless workflow execution. Serverless computing has emerged as a platform for scientific and data-intensive DAG workflows. Wukong [9] and Speedo [11] provide scalable dispatch frameworks for parallel workflows. Mashup [34] demonstrates that HPC DAG workflows can be executed efficiently through hybrid VM-serverless deployment, and DayDream [35] extends this direction to dynamic scientific workflows whose execution paths vary across runs, exploiting phase-level concurrency prediction and hot-start scheduling on pure serverless platforms. FaaS-based execution of HPC workflows is further validated in HyperFlow [28] and related systems [27]. ORION [25] and WiseFuse [26] optimize serverless DAG execution through DAG-aware resource sizing, function bundling, and workload characterization, using production traces from Azure Durable Functions. Jolteon [51] builds a stochastic performance model that, given a latency or cost constraint, configures resources to minimize the other. Dexter [31] tunes per-stage parallelism to optimize total runtime cost, and MinFlow [23] reduces the data-passing overhead of DAG shuffles through multi-level topologies. Netherite [7] demonstrates that state persistence and checkpointing are central to efficient serverless workflow execution, achieving order-of-magnitude throughput improvements through group commit and persistence pipelining. On the platform side, Bilal *et al.* [6] develop function-level resource allocation to exploit the flexibility serverless provides, Pronghorn [21] introduces checkpoint-based hot-start optimization that reduces cold-start overhead, Nightcore [17] efficiently serves latency-sensitive workloads on serverless runtimes, and Trident [52] addresses provider-side resource management for serverless platforms.

However, none of this body of work addresses the combination of serverless DAG execution with transient spot instance pools, leaving the fault tolerance challenges that arise from spot preemption in serverless workflows unexplored.

9 Conclusion

We presented FaaSpot, a serverless workflow execution system that addresses the cascade failures spot preemptions trigger in complex DAG workflows. Prior systems protect high-impact tasks by migrating them to reliable, non-spot resources, sacrificing spot's cost advantage. FaaSpot instead classifies each task once at submission time and shares this classification across its three online mechanisms, protecting the same high-impact positions on spot, without runtime coordination or migration. Our evaluation with eight DAG workflows on an in-house trace-driven spot emulator shows that FaaSpot reaches an 89.03% success rate at $0.018 cost per success, outperforming four state-of-the-art baselines on success rate, cost, and latency simultaneously.

References

[1] Ahsan Ali, Riccardo Pinciroli, Feng Yan, and Evgenia Smirni. 2022. Optimizing Inference Serving on Serverless Platforms. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2071–2084.
[2] Amazon Web Services. 2026. AWS Step Functions – workflow orchestration. https://aws.amazon.com/step-functions/.
[3] Apache OpenWhisk. 2026. Apache OpenWhisk Composer. https://github.com/apache/openwhisk-composer.
[4] Orna Agmon Ben-Yehuda, Muli Ben-Yehuda, Assaf Schuster, and Dan Tsafrir. 2013. Deconstructing Amazon EC2 Spot Instance Pricing. *ACM Transactions on Economics and Computation* 1, 3 (2013), 1–20.
[5] Shishir Bharathi, Ann Chervenak, Ewa Deelman, Gaurang Mehta, Mei-Hui Su, and Karan Vahi. 2008. Characterization of Scientific Workflows. In *The 2nd Workshop on Workflows in Support of Large-scale Science (WORKS)*.
[6] Muhammad Bilal, Marco Canini, Rodrigo Fonseca, and Rodrigo Rodrigues. 2023. With Great Freedom Comes Great Opportunity: Rethinking Resource Allocation for Serverless Functions. In *European Conference on Computer Systems (EuroSys)*.
[7] Sebastian Burckhardt, Badrish Chandramouli, Chris Gillum, David Justo, Konstantinos Kallas, Connor McMahon, Christopher Meiklejohn, and Xiangfeng Zhu. 2022. Netherite: Efficient Execution of Serverless Workflows. *Proceedings of the VLDB Endowment* 15, 8 (2022), 1591–1604.
[8] João Carreira, Pedro Fonseca, Alexey Tumanov, Andrew Zhang, and Randy H. Katz. 2019. Cirrus: a Serverless Framework for End-to-end ML Workflows. In *ACM Symposium on Cloud Computing (SoCC)*.
[9] Benjamin Carver, Jingyuan Zhang, Ao Wang, Ali Anwar, Panruo Wu, and Yue Cheng. 2020. Wukong: A Scalable and Locality-enhanced Framework for Serverless Parallel Computing. In *ACM Symposium on Cloud Computing (SoCC)*.
[10] Laura Clarke, Xiangqun Zheng-Bradley, Richard Smith, Eugene Kulesha, Chunlin Xiao, Iliana Toneva, Brendan Vaughan, Don Preuss, Rasko Leinonen, Martin Shumway, Stephen Sherry, Paul Flicek, et al. 2012. The 1000 Genomes Project: Data Management and Community Access. *Nature Methods* 9, 5 (2012), 459–462.
[11] Nilanjan Daw, Umesh Bellur, and Purushottam Kulkarni. 2021. Speedo: Fast Dispatch and Orchestration of Serverless Workflows. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*. ACM, 585–599. https://doi.org/10.1145/3472883.3486982
[12] Fission Project. 2026. Fission: Open source Kubernetes-native Serverless Framework. https://fission.io/.
[13] Google Cloud. 2026. Cloud Run Functions Quotas and Limits. https://docs.cloud.google.com/functions/quotas.
[14] Google Cloud. 2026. Workflows. https://cloud.google.com/workflows.
[15] Aaron Harlap, Andrew Chung, Alexey Tumanov, Gregory R. Ganger, and Phillip B. Gibbons. 2018. Tributary: Spot-dancing for Elastic Services with Latency SLOs. In *USENIX Annual Technical Conference (USENIX ATC)*.
[16] David Irwin, Prashant Shenoy, Pradeep Ambati, Prateek Sharma, Supreeth Shastri, and Ahmed Ali-Eldin. 2019. The Price Is (Not) Right: Reflections on Pricing for Transient Cloud Servers. In *International Conference on Computer Communications and Networks (ICCCN)*.
[17] Zhipeng Jia and Emmett Witchel. 2021. Nightcore: Efficient and Scalable Serverless Computing for Latency-sensitive, Interactive Microservices. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
[18] Pedro Joaquim, Manuel Bravo, Luís Rodrigues, and Miguel Matos. 2019. Hourglass: Leveraging Transient Resources for Time-Constrained Graph Processing in the Cloud. In *European Conference on Computer Systems (EuroSys)*.
[19] E. L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481.
[20] Kyunghwan Kim and Kyungyong Lee. 2024. Making Cloud Spot Instance Interruption Events Visible. In *ACM on Web Conference (WWW)*.
[21] Sumer Kohli, Shreyas Kharbanda, Rodrigo Bruno, Joao Carreira, and Pedro Fonseca. 2024. Pronghorn: Effective Checkpoint Orchestration for Serverless Hot-Starts. In *European Conference on Computer Systems (EuroSys)*.
[22] Sungjae Lee, Jaeil Hwang, and Kyungyong Lee. 2022. SpotLake: Diverse Spot Instance Dataset Archive Service. In *IEEE International Symposium on Workload Characterization (IISWC)*.
[23] Tao Li, Yongkun Li, Wenzhe Zhu, Yinlong Xu, and John C. S. Lui. 2024. MinFlow: High-performance and Cost-efficient Data Passing for I/O-intensive Stateful Serverless Analytics. In *USENIX Conference on File and Storage Technologies (FAST)*.
[24] Chengzhi Lu, Huanle Xu, Yudan Li, Wenyan Chen, Kejiang Ye, and Chengzhong Xu. 2024. SMIless: Serving DAG-based Inference with Dynamic Invocations under Serverless Computing. In *International Conference for High Performance Computing, Networking, Storage, and Analysis (SC)*.
[25] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. ORION and the Three Rights: Sizing, Bundling, and Prewarming for Serverless DAGs. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

[26] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Eshaan Minocha, Sameh Elnikety, Saurabh Bagchi, and Somali Chaterji. 2022. WISEFUSE: Workload Characterization and DAG Transformation for Serverless Workflows. In *ACM SIGMETRICS Conference (SIGMETRICS)*.

[27] Marcin Majewski, Maciej Pawlik, and Maciej Malawski. 2021. Algorithms for scheduling scientific workflows on serverless architecture. In *IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*.

[28] Maciej Malawski, Adam Gajek, Adam Zima, Bartosz Balis, and Kamil Figiela. 2020. Serverless Execution of Scientific Workflows: Experiments with HyperFlow, AWS Lambda and Google Cloud Functions. *Future Generation Computer Systems* 110 (2020), 502–514.

[29] Microsoft. 2026. Azure Functions Hosting Options. https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale.

[30] Microsoft. 2026. Durable Functions Overview. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview.

[31] Anna Maria Nestorov, Diego Marrón, Alberto Gutierrez-Torre, Chen Wang, Claudia Misale, Alaa Youssef, David Carrera, and Josep Lluís Berral. 2024. Dexter: A Performance-Cost Efficient Resource Allocation Manager for Serverless Data Analytics. In *ACM International Middleware Conference (MIDDLEWARE)*.

[32] Deepak Poola, Kotagiri Ramamohanarao, and Rajkumar Buyya. 2016. Enhancing Reliability of Workflow Execution Using Task Replication and Spot Instances. *ACM Transactions on Autonomous and Adaptive Systems* 10, 4 (2016), 21 pages.

[33] Felipe A. Portella, Paulo J. B. Estrela, Renzo Q. Malini, Luan Teylo, Josep L. Berral, and Lucia M. A. Drummond. 2023. MScheduler: Leveraging Spot Instances for High-Performance Reservoir Simulation in the Cloud. In *IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*.

[34] Rohan Basu Roy, Tirthak Patel, Vijay Gadepally, and Devesh Tiwari. 2022. Mashup: Making Serverless Computing Useful for HPC Workflows via Hybrid Execution. In *ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*.

[35] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2022. DayDream: Executing Dynamic Scientific Workflows on Serverless Platforms with Hot Starts. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[36] Amit Samanta, Ryan Stutsman, and Rohan Basu Roy. 2025. GridGreen: Integrating Serverless Computing in HPC Systems for Performance and Sustainability. In *ACM Symposium on Cloud Computing (SoCC)*.

[37] Prateek Sharma, Tian Guo, Xin He, David Irwin, and Prashant Shenoy. 2016. Flint: batch-interactive data-intensive processing on transient servers. In *European Conference on Computer Systems (EuroSys)*.

[38] Prateek Sharma, David Irwin, and Prashant Shenoy. 2017. Portfolio-driven Resource Management for Transient Cloud Servers. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, Article 5 (2017), 23 pages.

[39] Prateek Sharma, Stephen Lee, Tian Guo, David Irwin, and Prashant Shenoy. 2015. SpotCheck: Designing a Derivative IaaS Cloud on the Spot Market. In *European Conference on Computer Systems (EuroSys)*.

[40] Supreeth Shastri and David Irwin. 2018. Cloud Index Tracking: Enabling Predictable Costs in Cloud Spot Markets. In *ACM Symposium on Cloud Computing (SoCC)*.

[41] Myungjun Son, Gulsum Gudukbay Akbulut, and Mahmut Taylan Kandemir. 2024. SpotVerse: Optimizing Bioinformatics Workflows with Multi-Region Spot Instances in Galaxy and Beyond. In *ACM International Middleware Conference (MIDDLEWARE)*.

[42] Supreeth Subramanya, Amr Rizk, and David Irwin. 2016. Cloud Spot Markets are Not Sustainable: The Case for Transient Guarantees. In *USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.

[43] Luan Teylo, Luciana Arantes, Pierre Sens, and Lucia Maria de A. Drummond. 2023. Scheduling Bag-of-Tasks in Clouds Using Spot and Burstable Virtual Machines. *IEEE Transactions on Cloud Computing* 11, 1 (2023), 984–998.

[44] The Kubernetes Authors. 2026. Kubernetes. https://kubernetes.io/.

[45] H. Topcuoglu, S. Hariri, and Min-You Wu. 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274.

[46] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. 2024. Can't Be Late: Optimizing Spot Instance Savings under Deadlines. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[47] Ying Yan, Yanjie Gao, Yang Chen, Zhongxin Guo, Bole Chen, and Thomas Moscibroda. 2016. TR-Spark: Transient Computing for Big Data Analytics. In *ACM Symposium on Cloud Computing (SoCC)*.

[48] Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liqun Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Íñigo Goiri, Eli Cortez, Terry Yang, Victor Rühle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Snape: Reliable and Low-Cost Computing with Mixture of Spot and On-Demand VMs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[49] Youngseok Yang, Geon-Woo Kim, Won Wook Song, Yunseong Lee, Andrew Chung, Zhengping Qian, Brian Cho, and Byung-Gon Chun. 2017. Pado: A Data Processing Engine for Harnessing Transient Resources in Datacenters. In *European Conference on Computer Systems (EuroSys)*.

[50] John W. Young. 1974. A first order approximation to the optimum checkpoint interval. *Commun. ACM* 17, 9 (sep 1974), 530–531.

[51] Zili Zhang, Chao Jin, and Xin Jin. 2024. Jolteon: Unleashing the Promise of Serverless for Serverless Workflows. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

[52] Botao Zhu, Yifei Zhu, Chen Chen, and Linghe Kong. 2025. Trident: A Provider-Oriented Resource Management Framework for Serverless Computing Platforms. *IEEE Transactions on Services Computing* 18, 5 (2025), 3334–3347.